

```
from google.colab import drive
drive.mount('/content/gdrive')
```

Mounted at /content/gdrive

```
path = "/content/gdrive/MyDrive/machine_learning/pertemuan07"
```

1. Import Library

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

2. Load data

```
data = pd.read_csv(path + "/praktikum_mandiri/data/dataset_satelit.csv")
data.head()
```

	No	Longitude	Latitude	N	P	K	Ca	Mg	Fe	Mn	...	b1	Sigma_VV	Sigma_VH	plia	lia	i
0	1	103.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.96	463.23	...	0.0433	0.18183	0.04461	35.74446	35.79744	35.41
1	2	103.037201	-0.604689	2.75	0.17	0.568	0.76	0.58	102.63	493.81	...	0.0465	0.22079	0.04640	35.12096	35.14591	35.41
2	3	103.036359	-0.603012	1.77	0.12	0.339	0.49	0.6	107.37	460.93	...	0.0417	0.18926	0.03992	35.07724	35.07730	35.41
3	4	103.036950	-0.603219	2.30	0.15	0.460	0.74	0.67	96.02	338.17	...	0.0367	0.14769	0.03622	36.08078	36.08469	35.41
4	5	103.036802	-0.601969	2.05	0.14	0.308	0.64	0.72	87.01	384.33	...	0.0361	0.18205	0.03797	32.68855	32.69293	35.41

5 rows × 34 columns

N, P, K, Ca, Mg, Fe, Mn: Ini kemungkinan adalah konsentrasi unsur hara tanah, seperti Nitrogen, Fosfor, Kalium, Kalsium, Magnesium, Besi, dan Mangan.

b1, Sigma_VV, Sigma_VH, plia, iafe, gamaO_vv, gamaO_vh, betaO_vv, betaO_vh: Ini adalah variabel teknis yang biasanya berasal dari data citra satelit, khususnya dari sensor radar (seperti Sentinel-1, ditunjukkan oleh Sigma_VV, Sigma_VH yang merupakan koefisien hamburan balik polarisasi Ganda Vertikal-Vertikal dan Vertikal-Horizontal). Variabel lain (b1, plia, iafe, dll.) mungkin merupakan pita spektral atau indeks yang dihitung dari citra optik atau data radar lainnya.

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 594 entries, 0 to 593
Data columns (total 34 columns):
#   Column      Non-Null Count  Dtype
---  -
0   No           594 non-null    int64
1   Longitude    594 non-null    float64
2   Latitude     594 non-null    float64
3   N            594 non-null    float64
4   P            594 non-null    float64
5   K            593 non-null    float64
6   Ca           594 non-null    float64
7   Mg           594 non-null    object
8   Fe           594 non-null    float64
9   Mn           594 non-null    float64
10  Cu           594 non-null    float64
11  Zn           594 non-null    float64
12  B            594 non-null    float64
13  b12          594 non-null    float64
14  b11          594 non-null    float64
15  b9           594 non-null    float64
16  b8a          594 non-null    float64
17  b8           594 non-null    float64
18  b7           594 non-null    float64
19  b6           594 non-null    float64
20  b5           594 non-null    float64
```

```

21 b4          594 non-null float64
22 b3          594 non-null float64
23 b2          594 non-null float64
24 b1          594 non-null float64
25 Sigma_VV    594 non-null float64
26 Sigma_VH    594 non-null float64
27 plia        594 non-null float64
28 lia         594 non-null float64
29 iafe        594 non-null float64
30 gamma0_vv   594 non-null float64
31 gamma0_vh   594 non-null float64
32 beta0_vv    594 non-null float64
33 beta0_vh    594 non-null float64
dtypes: float64(32), int64(1), object(1)
memory usage: 157.9+ KB

```

data.info() merupakan sebuah code untuk memberikan ringkasan komprehensif tentang dataframe

```
data.describe()
```

	No	Longitude	Latitude	N	P	K	Ca	Fe	Mn	Cu	...
count	594.000000	594.000000	594.000000	594.000000	594.000000	593.000000	594.000000	594.000000	594.000000	594.000000	...
mean	297.500000	106.878644	-1.024933	2.259091	0.141380	0.582175	0.595094	74.613771	308.034697	2.391195	...
std	171.617307	4.949840	0.965349	0.395499	0.019782	0.222567	0.366118	55.579655	241.731643	1.580296	...
min	1.000000	102.760857	-2.333750	1.140000	0.090000	0.122000	0.050000	21.080000	3.160000	0.090000	...
25%	149.250000	102.927811	-2.233338	1.982500	0.130000	0.429000	0.320000	40.705000	124.015000	1.172500	...
50%	297.500000	103.581969	-0.602276	2.280000	0.140000	0.549000	0.540000	65.650000	239.445000	2.225000	...
75%	445.750000	113.403797	-0.257349	2.570000	0.150000	0.710000	0.790000	87.372500	434.990000	3.357500	...
max	594.000000	113.434700	0.069251	3.230000	0.220000	1.489000	2.820000	559.100000	2009.320000	8.170000	...

8 rows × 33 columns

data.describe() merupakan code untuk menghasilkan statistik deskriptif dari data. Metode ini secara otomatis memilih semua kolom yang berisi data numerik (int atau float) dan menghitung ringkasan statistik.

Metrik, Nama Lengkap, Penjelasan

1. count = Menunjukkan jumlah total entri yang tidak hilang (non-missing) dalam kolom tersebut. Nilai ini penting untuk memastikan tidak ada nilai yang hilang (NaN) di kolom. Dalam output Anda, 594.000000 berarti ada 594 baris data yang valid.
2. mean = Nilai rata-rata aritmatika dari semua angka dalam kolom. Dihitung dengan menjumlahkan semua nilai dan membaginya dengan jumlah total nilai. Ini adalah ukuran kecenderungan sentral (pusat data).
3. std = Mengukur penyebaran atau variasi nilai data dari nilai rata-rata (mean). Standar Deviasi yang tinggi menunjukkan bahwa data tersebar luas (bervariasi tinggi), sedangkan nilai yang rendah menunjukkan data cenderung dekat dengan rata-rata.
4. min = Nilai terkecil atau terendah yang ada dalam kolom.
5. 25% = Nilai di mana 25% data berada di bawahnya. Kuartil ini menandai batas bawah dari Middle 50% data (bersama dengan Q3).
6. 50% = Nilai tengah dari data ketika data sudah diurutkan. 50% data berada di bawah nilai ini. Median sering digunakan daripada rata-rata jika data mengandung outliers (pencilan), karena median kurang sensitif terhadap nilai ekstrem.
7. 75% = Nilai di mana 75% data berada di bawahnya (atau 25% data berada di atasnya). Kuartil ini menandai batas atas dari Middle 50% data.
8. max = Nilai terbesar atau tertinggi yang ada dalam kolom.

```
print(data.columns)
```

```

Index(['No', 'Longitude', 'Latitude', 'N', 'P', 'K', 'Ca', 'Mg', 'Fe', 'Mn',
      'Cu', 'Zn', 'B', 'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
      'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
      'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh'],
      dtype='object')

```

data.columns adalah mengambil daftar semua nama kolom yang ada dalam DataFrame tersebut.

3. Feature Selection

```
data['Mg'] = pd.to_numeric(data['Mg'], errors='coerce')
data= data.dropna()
```

```
# X = data[['b2', 'b3', 'b4', 'b8', 'b11', 'Sigma_VV', 'Sigma_VH']]
# y = data['N']
```

memilih variabel X dan Y

```
X = data[['P', 'K', 'Ca', 'Mg', 'Fe', 'Mn',
          'Cu', 'Zn', 'B', 'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
          'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
          'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh']]
y = data['N']
```

memilih variabel X dan Y

4. Splitting Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print("Jumlah data training:", len(X_train))
print("Jumlah data testing:", len(X_test))
```

```
Jumlah data training: 473
Jumlah data testing: 119
```

- `train_test_split(...)` = fungsi dari pustaka Scikit-learn (atau sklearn), yang digunakan untuk membagi data.
- `X, Y`, = `X` adalah matriks fitur (features) atau data independen (semua variabel satelit dan unsur hara). `Y` adalah vektor target atau data dependen (variabel yang ingin di prediksi).
- `test_size=0.2` = Menentukan proporsi data yang akan dialokasikan untuk set pengujian. Di sini, 20% (≈ 0.2) dari total data akan digunakan untuk pengujian, dan sisanya 80% untuk pelatihan.
- `random_state=42` = Menentukan seed (benih) untuk pengacakan. Menggunakan `random_state` memastikan bahwa setiap kali kode ini dijalankan, data akan selalu dibagi dengan cara yang sama persis. Angka 42 hanyalah konvensi umum.
- `X_train, X_test, y_train, y_test` = Output dari fungsi `train_test_split` disimpan ke empat variabel:
- `X_train` & `y_train`: Data fitur dan target untuk Pelatihan (80%).
- `X_test` & `y_test`: Data fitur dan target untuk Pengujian (20%).

5. Modelling

```
model = LinearRegression()
model.fit(X_train, y_train)
```

LinearRegression ⓘ ?

```
LinearRegression()
```

```
#testing
y_pred = model.predict(X_test)

#evaluasi model
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("R Score", r2)
print("RMSE", rmse)
```

```
R Score 0.7549955124484078
RMSE 0.19392193397227142
```

- `y_pred = model.predict(X_test)` = Menggunakan model yang sudah dilatih (model) untuk membuat prediksi pada data fitur pengujian (`X_test`). Hasil prediksi ini, yaitu nilai target yang diprediksi oleh model, disimpan dalam variabel `y_pred`.
- `r2 = r2_score(y_test, y_pred)` = Menghitung R^2 Score (Coefficient of Determination), sebuah metrik yang digunakan untuk regresi.
- `mse = mean_squared_error(y_test, y_pred)` = Menghitung MSE (Mean Squared Error). Ini mengukur rata-rata kuadrat dari perbedaan antara nilai yang diprediksi dan nilai aktual.
- `rmse = np.sqrt(mse)` = Menghitung RMSE (Root Mean Squared Error). RMSE adalah akar kuadrat dari MSE. Ini lebih mudah diinterpretasikan karena memiliki satuan yang sama dengan variabel target (Y).

```
coeff = pd.DataFrame({
    'Fitur' : X.columns,
    'Koefisien' : model.coef_
})
print(coeff)
```

	Fitur	Koefisien
0	P	8.570837
1	K	0.056014
2	Ca	-0.058380
3	Mg	-0.161095
4	Fe	0.000284
5	Mn	0.000177
6	Cu	0.023202
7	Zn	0.002968
8	B	-0.002092
9	b12	0.327312
10	b11	-0.619437
11	b9	-0.047586
12	b8a	0.064453
13	b8	-0.072659
14	b7	0.981186
15	b6	-0.864947
16	b5	-0.329397
17	b4	1.420614
18	b3	-0.449043
19	b2	-0.357454
20	b1	-0.033418
21	Sigma_VV	0.156595
22	Sigma_VH	-1.466200
23	plia	0.021315
24	lia	-0.021906
25	iafe	-0.078122
26	gamma0_vv	0.457327
27	gamma0_vh	2.946416
28	beta0_vv	-0.569281
29	beta0_vh	-0.677599

- `model.coef_` = attribute penting dari model regresi linier (seperti Linear Regression atau Ridge/Lasso) di Scikit-learn. Ini menyimpan nilai koefisien regresi yang dihitung model untuk setiap fitur (feature) input. Koefisien menunjukkan bobot atau pengaruh setiap fitur terhadap variabel target.
- `pd.DataFrame({...})` = Membuat objek DataFrame Pandas untuk mengatur hasil secara tabular.
- `'Fitur' : X.columns` = Kolom pertama DataFrame diberi nama 'Fitur'. Nilainya diambil dari `X.columns`, yaitu daftar nama kolom (feature) yang digunakan untuk melatih model.
- `'Koefisien' : model.coef_` = Kolom kedua diberi nama 'Koefisien'. Nilainya diambil dari `model.coef_`, yang merupakan bobot numerik dari setiap fitur.
- `coeff = ...` = DataFrame yang dibuat disimpan dalam variabel bernama `coeff`
- `print(coeff)` = Mencetak DataFrame `coeff` yang berisi pasangan fitur dan koefisiennya.

```
import statsmodels.api as sm
X_sm = sm.add_constant(X)
model_ols = sm.OLS(y, X_sm).fit()
print(model_ols.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          N      R-squared:                0.779
Model:                  OLS    Adj. R-squared:           0.768
Method:                 Least Squares    F-statistic:       66.10
Date:                   Mon, 03 Nov 2025    Prob (F-statistic):  6.69e-163
Time:                   08:16:32    Log-Likelihood:     156.68
No. Observations:       592    AIC:                  -251.4
```

```

Df Residuals:      561    BIC:      -115.5
Df Model:          30
Covariance Type:  nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
const          3.8681      0.864      4.479      0.000      2.172      5.564
P              8.5760      0.465     18.428      0.000      7.662      9.490
K              0.0273      0.051      0.530      0.596     -0.074      0.128
Ca            -0.0416      0.023     -1.808      0.071     -0.087      0.004
Mg            -0.2166      0.063     -3.464      0.001     -0.339     -0.094
Fe             0.0002      0.000      1.102      0.271     -0.000      0.001
Mn             0.0002    4.59e-05      4.083      0.000    9.72e-05      0.000
Cu             0.0229      0.006      4.046      0.000      0.012      0.034
Zn            -0.0024      0.001      1.889      0.059   -9.52e-05      0.005
B            -0.0022      0.001     -1.476      0.141     -0.005      0.001
b12           0.2010      0.534      0.377      0.707     -0.848      1.250
b11          -0.9945      0.517     -1.922      0.055     -2.011      0.022
b9           -0.0802      0.067     -1.191      0.234     -0.212      0.052
b8a           0.0962      0.067      1.427      0.154     -0.036      0.229
b8           -0.0806      0.024     -3.362      0.001     -0.128     -0.034
b7            1.3059      0.542      2.409      0.016      0.241      2.371
b6           -0.9238      0.462     -2.001      0.046     -1.831     -0.017
b5           -0.5470      0.488     -1.120      0.263     -1.506      0.412
b4            1.2561      0.972      1.292      0.197     -0.653      3.166
b3            0.3085      1.639      0.188      0.851     -2.910      3.527
b2           -0.7075      0.942     -0.751      0.453     -2.559      1.144
b1            0.0023      0.352      0.006      0.995     -0.689      0.694
Sigma_VV       0.9543      0.889      1.074      0.283     -0.792      2.700
Sigma_VH      -1.1666      0.861     -1.356      0.176     -2.857      0.524
plia          -0.0113      0.071     -0.159      0.873     -0.150      0.128
lia           0.0102      0.071      0.144      0.886     -0.129      0.149
iafe          -0.0802      0.023     -3.475      0.001     -0.126     -0.035
gamma0_vv     -0.5040      2.180     -0.231      0.817     -4.785      3.777
gamma0_vh      3.1827      4.643      0.685      0.493     -5.938     12.303
beta0_vv      -0.2381      1.580     -0.151      0.880     -3.342      2.865
beta0_vh      -0.5472      3.536     -0.155      0.877     -7.493      6.399
=====
Omnibus:          28.388    Durbin-Watson:          1.527
Prob(Omnibus):    0.000    Jarque-Bera (JB):        53.216
Skew:             -0.310    Prob(JB):                2.78e-12
Kurtosis:         4.332    Cond. No.:               2.98e+05
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.98e+05. This might indicate that there are strong multicollinearity or other numerical problems.